

Missing Data in SEM

SEM in R using Lavaan



**Utrecht
University**

Kyle M. Lang

Department of Methodology & Statistics
Utrecht University

Outline

Missing Data Mechanisms

Imputation-Based Solutions

- Single Imputation

- Multiple Imputation

- MI-Based Analysis

ML-Based Solutions

- Maximum Likelihood Estimation

- Full Information Maximum Likelihood

Auxiliary Variables



What are Missing Data?

Missing data are empty cells in a dataset where there should be observed values.

- The missing cells correspond to true population values, but we haven't observed those values.



What are Missing Data?

Missing data are empty cells in a dataset where there should be observed values.

- The missing cells correspond to true population values, but we haven't observed those values.

Not every empty cell is a missing datum.

- Quality-of-life ratings for dead patients in a mortality study
- Firm profitability after the company goes out of business
- Self-reported severity of menstrual cramping for men
- Empty blocks of data following “gateway” items

A Little Notation

$Y :=$ An $N \times P$ Matrix of Arbitrary Data

$Y_{mis} :=$ The *missing* part of Y

$Y_{obs} :=$ The *observed* part of Y

$R :=$ An $N \times P$ response matrix

$M :=$ An $N \times P$ missingness matrix

The R and M matrices are complementary.

- $r_{np} = 1$ means y_{np} is observed; $m_{np} = 1$ means y_{np} is missing.
- $r_{np} = 0$ means y_{np} is missing; $m_{np} = 0$ means y_{np} is observed.
- M_p is the *missingness* of Y_p .

Example

```
## Load some useful packages:
library(dplyr)
library(naniar)
library(ggmice)

## Read in some data:
bfi0 <- readRDS(here::here("data", "bfi_datasets.rds"))
bfi <- bfi0$incomplete |> select(-matches("N\\d|C\\d|E\\d|male"))

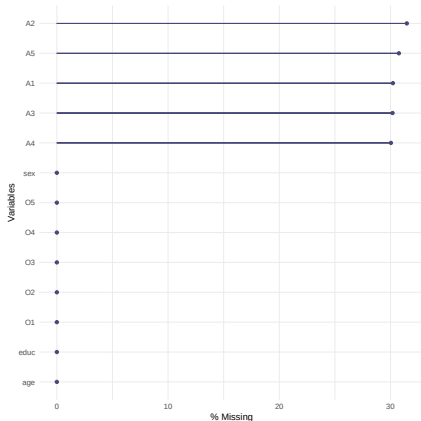
## Compute the variablewise proportions of missing data:
bfi |> is.na() |> colMeans() |> round(2)
```

A1	A2	A3	A4	A5	01	02	03	04	05	age	sex	educ
0.30	0.31	0.30	0.30	0.31	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00

Example

Visualize the percentages missing via **naniar::gg_miss_var()**.

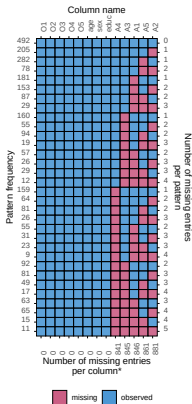
```
gg_miss_var(bfi, show_pct = TRUE)
```



Example

Visualize the missing data patterns via `ggmice::plot_pattern()`.

```
plot_pattern(bfi, rotate = TRUE)
```



*total number of missing entries: 4274

Example

In **lavaan**, we can directly fit a model with incomplete data.

```
library(lavaan)

## Specify the measurement model:
cfaMod <- '
agree =~ A1 + A2 + A3 + A4 + A5
open  =~ O1 + O2 + O3 + O4 + O5
'

## Estimate the model:
naiveOut <- cfa(cfaMod, data = bfi, std.lv = TRUE)
```

Example

The model will estimate, just fine...

```
lavaan 0.6-21 ended normally after 21 iterations
```

Latent Variables:

	Estimate	Std.Err	z-value	P(> z)
agree =~				
A1	0.561	0.071	7.866	0.000
A2	-0.725	0.056	-13.055	0.000
A3	-0.901	0.063	-14.383	0.000
A4	-0.599	0.074	-8.103	0.000
A5	-0.781	0.059	-13.264	0.000
open =~				
O1	0.490	0.055	8.983	0.000
O2	-0.844	0.079	-10.649	0.000
O3	0.786	0.058	13.574	0.000
O4	0.311	0.055	5.699	0.000
O5	-0.874	0.073	-11.987	0.000

Example

Covariances:

	Estimate	Std.Err	z-value	P(> z)
agree ~~				
open	-0.254	0.060	-4.204	0.000

Variances:

	Estimate	Std.Err	z-value	P(> z)
.A1	1.662	0.114	14.593	0.000
.A2	0.781	0.067	11.650	0.000
.A3	0.865	0.086	10.050	0.000
.A4	1.776	0.122	14.517	0.000
.A5	0.863	0.075	11.430	0.000
.01	0.880	0.064	13.823	0.000
.02	1.704	0.133	12.776	0.000
.03	0.658	0.072	9.150	0.000
.04	0.976	0.065	15.020	0.000
.05	1.288	0.112	11.460	0.000

Example

But not everything is as it seems.

	Used	Total
Number of observations	492	2800



Default Approach

Like most software packages, **lavaan** will default to *complete case analysis* when asked to analyze incomplete data.

- In the absence of user input, this is a sensible option.
- That doesn't mean you should actually use deletion to treat the missing data in your analysis.



Default Approach

Like most software packages, **lavaan** will default to *complete case analysis* when asked to analyze incomplete data.

- In the absence of user input, this is a sensible option.
- That doesn't mean you should actually use deletion to treat the missing data in your analysis.

Complete case analysis has two major problems.

1. Throws out useful information (potentially a lot of information)
2. Probably biases parameter estimates.

To understand the second point, we need to discuss *missing data mechanisms*.

MISSING DATA MECHANISMS



Missing Data Mechanisms

Missing Completely at Random (MCAR)

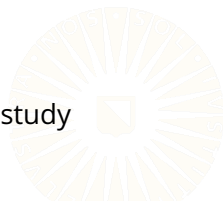
- $P(R|Y_{mis}, Y_{obs}) = P(R)$
- Missingness is unrelated to any study variables.

Missing at Random (MAR)

- $P(R|Y_{mis}, Y_{obs}) = P(R|Y_{obs})$
- Missingness is related to only the *observed* parts of study variables.

Missing not at Random (MNAR)

- $P(R|Y_{mis}, Y_{obs}) \neq P(R|Y_{obs})$
- Missingness is related to the *unobserved* parts of study variables.



Simulate Some Toy Data

```
nObs <- 5000 # Sample Size
pm    <- 0.3  # Proportion Missing

sigma <- matrix(c(1.0, 0.5, 0.3,
                  0.5, 1.0, 0.0,
                  0.3, 0.0, 1.0),
                ncol = 3)
tmp <- rmvnorm(nObs, c(0, 0, 0), sigma)

x0 <- tmp[, 1]
y0 <- tmp[, 2]
z0 <- tmp[, 3]

cor(y0, x0) # Check correlation between X and Y

[1] 0.4997145
```

MCAR Example

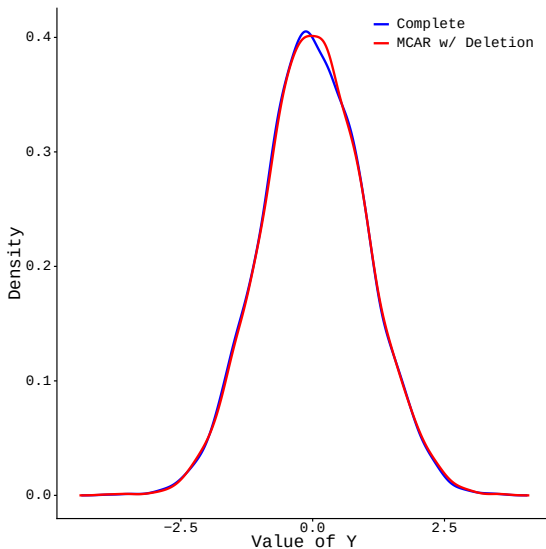
```
## Simulate MCAR Missingness:
mVec <- sample(1 : length(y0), size = pm * length(y0))

yMcar      <- y0
yMcar[mVec] <- NA

cor(yMcar, x0, use = "pairwise") # Look at correlation

[1] 0.5195767
```

MCAR Example



MAR Example

```
## Simulate MAR Missingness:
mVec <- x0 < quantile(x0, probs = pm)
mean(mVec)

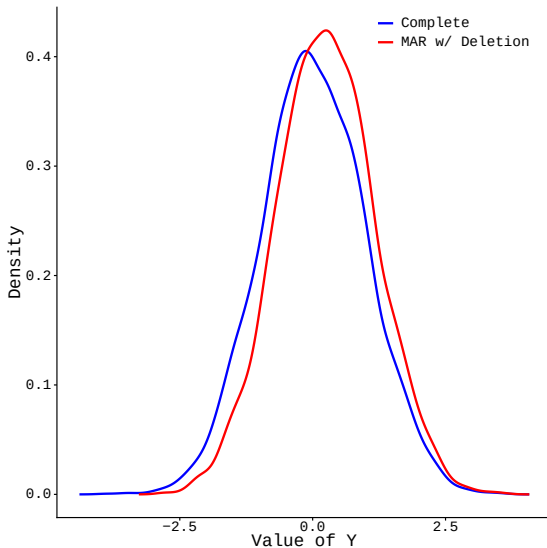
[1] 0.3

yMar      <- y0
yMar[mVec] <- NA

cor(yMar, x0, use = "pairwise") # Not looking so good :(

[1] 0.3822143
```

MAR Example



MNAR Example

```
## Simulate MNAR Missingness:
mVec <- y0 < quantile(y0, probs = pm)
mean(mVec)

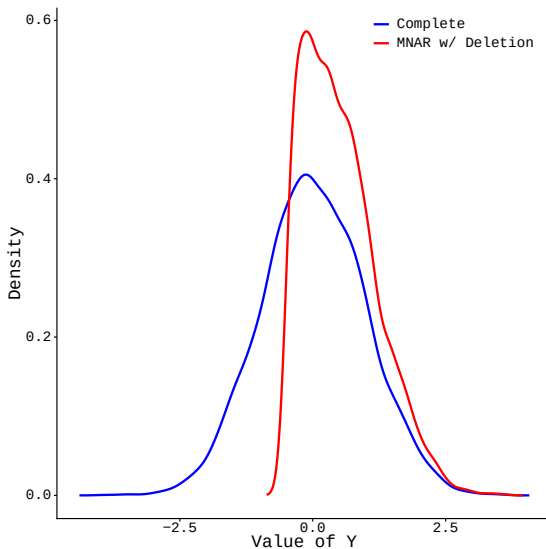
[1] 0.3

yMnar      <- y0
yMnar[mVec] <- NA

cor(yMnar, x0, use = "pairwise") # Hmm...looks pretty bad.

[1] 0.3902962
```

MNAR Example



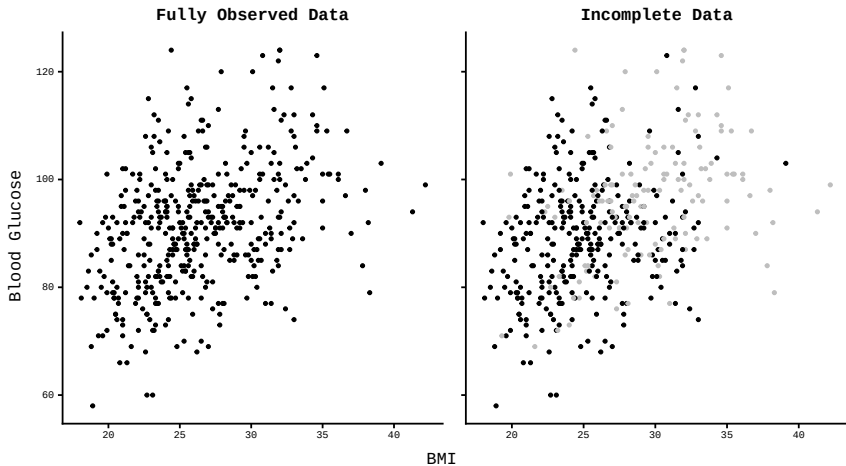
Effects of Deletion

As we saw in the preceding plots, excluding incomplete cases usually alters the variables' distributions.

- The statistics upon which we base our analyses generally summarize these distributions.
- Problems with the distributions show up as bias in the results of our analyses.

```
diabetes1 <- diabetes2 <-  
  readRDS(here::here("data", "diabetes.rds"))  
  
mVec <- simLogisticMissingness0(data      = diabetes1,  
                                pm        = 0.3,  
                                preds     = c("bmi", "bp"),  
                                stdData   = TRUE)$r  
  
diabetes2[mVec, "glu"] <- NA
```


Example



Example

```
diabetes1 |> select(bmi, glu, bp) |> cor()
```

	bmi	glu	bp
bmi	1.0000000	0.38868	0.3954109
glu	0.3886800	1.00000	0.3904300
bp	0.3954109	0.39043	1.0000000

```
diabetes2 |> select(bmi, glu, bp) |> cor(use = "complete")
```

	bmi	glu	bp
bmi	1.0000000	0.2566595	0.2052338
glu	0.2566595	1.0000000	0.3011547
bp	0.2052338	0.3011547	1.0000000

Example

```
mean(diabetes1$glu)
```

```
[1] 91.26018
```

```
mean(diabetes2$glu, na.rm = TRUE)
```

```
[1] 88.86424
```

```
var(diabetes1$glu)
```

```
[1] 132.1657
```

```
var(diabetes2$glu, na.rm = TRUE)
```

```
[1] 115.8254
```

Good Missing Data Methods

Multiple Imputation (MI)

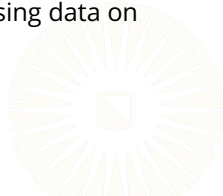
- Replace the missing values with M plausible estimates
 - Essentially, a repeated application of stochastic, regression-based prediction to synthesize replacements for the missing data.
 - Produces unbiased parameter estimates and predictions
 - Produces “correct” standard errors, CIs, and prediction intervals
 - Very, very flexible
 - Computationally expensive



Good Missing Data Methods

Full Information Maximum Likelihood (FIML)

- Adjust the objective function to only consider the observed parts of the data
 - Models are directly estimated in the presence of missing data
 - The predictors of nonresponse must be included in the model, somehow
 - FIML is only available for certain types of models
 - In linear regression models, FIML cannot treat missing data on predictors (if the predictors are taken as fixed)



Good Missing Data Methods

What happens when we apply MI to our previous MAR example?

```
## Estimate imputation model:
miceOut <- mice(data      = data.frame(y = yMar, x = x0),
               m          = 25,
               maxit      = 1,
               method     = "norm",
               printFlag = FALSE)

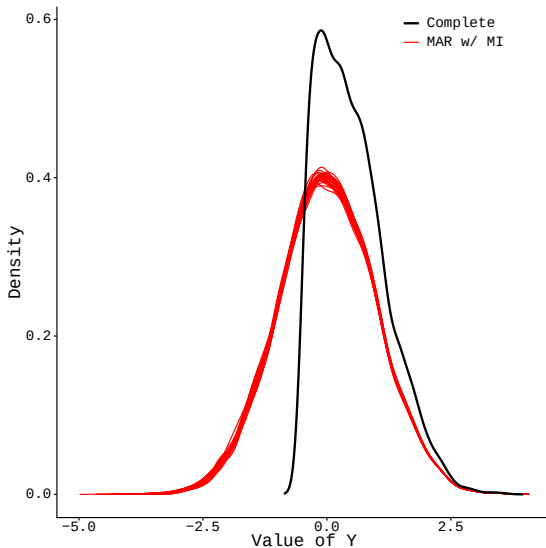
## Estimate and pool M correlations:
with(miceOut, cor(y, x))$analyses |> unlist() |> mean()

[1] 0.504661
```

The MI-based parameter estimate looks good.

- MI produces unbiased estimates when data are MAR.

Good Missing Data Methods



Good Missing Data Methods

What about applying MI to our MNAR example?

```
## Estimate imputation model:
miceOut <- mice(data      = data.frame(y = yMnar, x = x0),
               m          = 25,
               maxit      = 1,
               method     = "norm",
               printFlag  = FALSE)

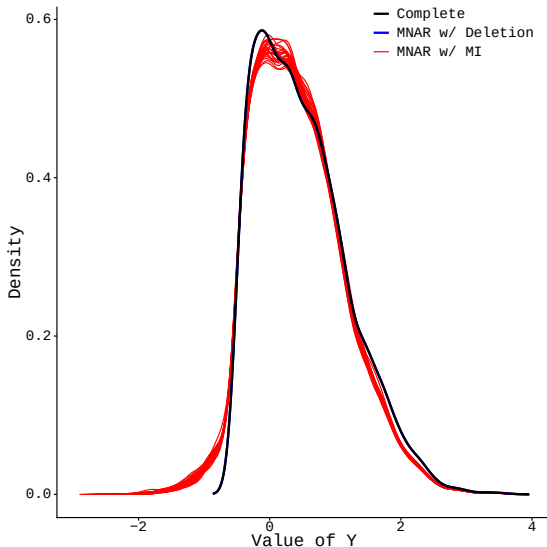
## Estimate and pool M correlations:
with(miceOut, cor(y, x))$analyses |> unlist() |> mean()

[1] 0.4116519
```

The MI-based parameter estimate is still biased.

- MI cannot correct bias when data are MNAR.

Good Missing Data Methods



MI Example

```
## The mice package does MI:  
library(mice)  
  
## Multiply impute the missing data:  
miceOut <- mice(data      = diabetes2,  
                 m        = 25,  
                 maxit    = 1,  
                 printFlag = FALSE,  
                 method   = "norm")
```

MI Example

```
## Complete data:
```

```
diabetes1 |> select(bmi, glu, bp) |> cor()
```

	bmi	glu	bp
bmi	1.0000000	0.38868	0.3954109
glu	0.3886800	1.00000	0.3904300
bp	0.3954109	0.39043	1.0000000

```
## MI:
```

```
pooledCorMat(miceOut, c("bmi", "glu", "bp"))
```

	bmi	glu	bp
bmi	1.0000000	0.3141910	0.3954109
glu	0.3141910	1.0000000	0.3630282
bp	0.3954109	0.3630282	1.0000000

MI Example

```
mean(diabetes1$glu)
```

```
[1] 91.26018
```

```
with(miceOut, mean(glu))$analyses |> unlist() |> mean()
```

```
[1] 90.64808
```

```
var(diabetes1$glu)
```

```
[1] 132.1657
```

```
with(miceOut, var(glu))$analyses |> unlist() |> mean()
```

```
[1] 122.3849
```

FIML Example

```
fit <- diabetes2 |>
  select(bmi, glu, bp) |>
  lavCor(missing = "fiml", output = "sampstat")

## Complete data:
diabetes1 |> summarize(mean = mean(glu), var = var(glu))

      mean      var
1 91.26018 132.1657

## FIML:
fit %$% c(mean = mean[["glu"]], var = cov[["glu", "glu"]])

      mean      var
90.82487 125.27146
```

FIML Example

```
## Complete data:
```

```
diabetes1 |> select(bmi, glu, bp) |> cor() |> round(3)
```

```
      bmi    glu    bp  
bmi 1.000 0.389 0.395  
glu 0.389 1.000 0.390  
bp  0.395 0.390 1.000
```

```
## FIML:
```

```
fit$cov |> cov2cor()
```

```
      bmi    glu    bp  
bmi 1.000  
glu 0.357 1.000  
bp  0.395 0.386 1.000
```

IMPUTATION-BASED SOLUTIONS



Prediction Example

To fix ideas, let's consider the following model:

$$Y_{LDL} = \beta_0 + \beta_1 X_{BP} + \beta_2 X_{gluc} + \beta_3 X_{BMI} + \varepsilon$$

Training this model on the first $N = 400$ observations from the *diabetes* data produces the following fitted model:

$$Y_{LDL} = 22.135 + 0.089X_{BP} + 0.498X_{gluc} + 1.48X_{BMI}$$



Prediction Example

To fix ideas, let's consider the following model:

$$Y_{LDL} = \beta_0 + \beta_1 X_{BP} + \beta_2 X_{gluc} + \beta_3 X_{BMI} + \varepsilon$$

Training this model on the first $N = 400$ observations from the *diabetes* data produces the following fitted model:

$$Y_{LDL} = 22.135 + 0.089X_{BP} + 0.498X_{gluc} + 1.48X_{BMI}$$

Suppose a new patient presents with $BP = 121$, $gluc = 89$, and $BMI = 30.6$. We can predict their *LDL* score by:

$$\begin{aligned}\hat{Y}_{LDL} &= 22.135 + 0.089(121) + 0.498(89) + 1.48(30.6) \\ &= 122.463\end{aligned}$$

Imputation is Just Prediction*

Imputation is nothing more than a type of prediction.

1. Train a model on the observed parts of the data, Y_{obs} .
 - Train the imputation model.
2. Predict the missing values, Y_{mis} .
 - Generate imputations.
3. Replace the missing values with these predictions.
 - Impute the missing data.



*Levels of Uncertainty Modeling

van Buuren (2018) classifies different imputation methods:

1. Simple Prediction

- The missing data are naively filled with predicted values from some regression equation.
- All uncertainty is ignored.

2. Prediction + Noise

- A random residual error is added to each predicted value to create the imputations.
- Only uncertainty in the predicted values is modeled.
- The imputation model itself is assumed to be correct and error-free.

3. Prediction + Noise + Model Error

- Uncertainty in the imputation model itself is also modeled.
- Required for proper imputations in the sense of Rubin (1987).

Do we really need to worry?

The arguments against single imputation can seem archaic and petty. Do we really need to worry about this stuff?



Do we really need to worry?

The arguments against single imputation can seem archaic and petty. Do we really need to worry about this stuff?

- YES!!! (At least if you care about inference)

The following are results from a simple Monte Carlo simulation:

	Complete Data	Conditional Mean	Stochastic	MI
cor(X, Y)	0.500	0.563	0.498	0.497
Type I Error	0.052	0.138	0.120	0.054



Do we really need to worry?

The arguments against single imputation can seem archaic and petty. Do we really need to worry about this stuff?

- YES!!! (At least if you care about inference)

The following are results from a simple Monte Carlo simulation:

	Complete Data	Conditional Mean	Stochastic	MI
cor(X, Y)	0.500	0.563	0.498	0.497
Type I Error	0.052	0.138	0.120	0.054

- Conditional mean substitution overestimates the correlation.
- Both single imputation methods inflate Type I error rates.
- MI provides unbiased estimates and accurate Type I error rates.

Simulate Some Toy Data

```
nObs <- 1000 # Sample Size
pm   <- 0.3  # Proportion Missing

sigma <- matrix(c(1.0, 0.5, 0.0,
                  0.5, 1.0, 0.3,
                  0.0, 0.3, 1.0),
               ncol = 3)

dat0 <- as.data.frame(rmvnorm(nObs, c(0, 0, 0), sigma))
colnames(dat0) <- c("y", "x", "z")
```

Simulate Some Toy Data

```
## Impose MAR Nonresponse:
dat1 <- dat0
mVec <- with(dat1, x < quantile(x, probs = pm))

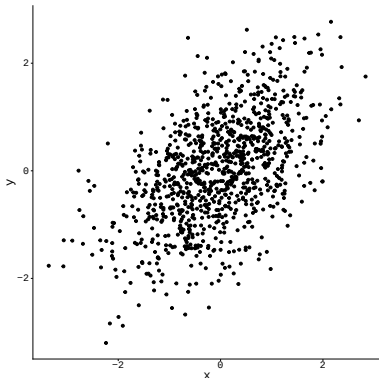
dat1[mVec, "y"] <- NA

## Subset the data:
yMis <- dat1[mVec, ]
yObs <- dat1[!mVec, ]
```


Look at the Data

```
round(head(dat0, n = 5), 3)
```

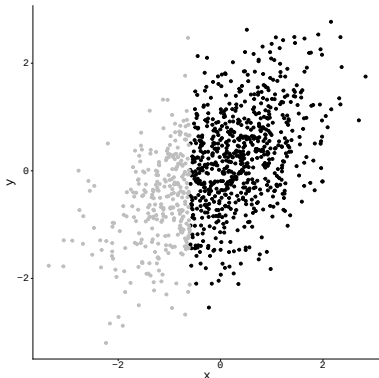
	y	x	z
1	0.094	-0.743	0.191
2	1.666	0.542	-0.181
3	0.208	0.617	0.711
4	0.133	-0.827	0.430
5	-0.003	0.441	0.719



Look at the Data

```
round(head(dat1, n = 5), 3)
```

	y	x	z
1	NA	-0.743	0.191
2	1.666	0.542	-0.181
3	0.208	0.617	0.711
4	NA	-0.827	0.430
5	-0.003	0.441	0.719



Expected Imputation Model Parameters

```
lsFit <- lm(y ~ x, data = y0bs)

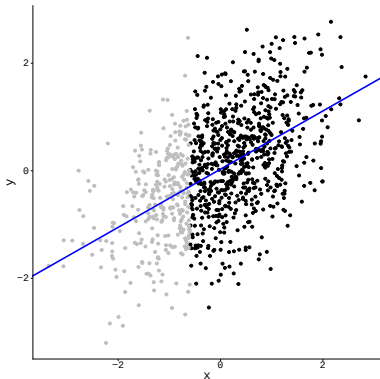
beta <- coef(lsFit)
sigma <- summary(lsFit)$sigma

as.matrix(beta)

              [,1]
(Intercept) 0.03113366
x            0.54074458

sigma

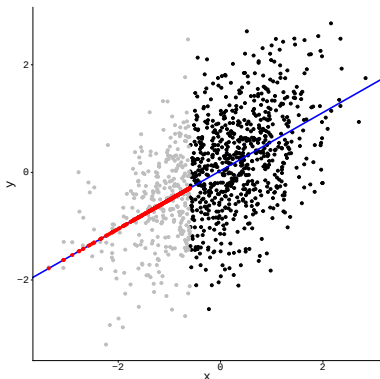
[1] 0.8535355
```



Conditional Mean Substitution

```
## Generate imputations:  
imps <- beta[1] +  
  beta[2] * yMis[ , "x"]  
  
## Fill missing cells in Y:  
dat1[mVec, "y"] <- imps  
  
round(head(dat1, n = 5), 3)
```

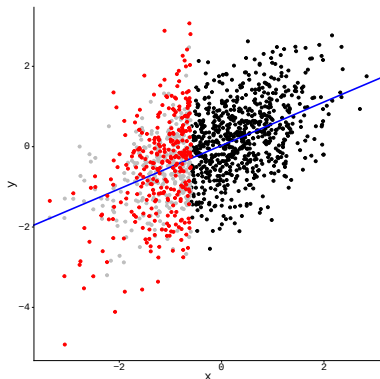
	y	x	z
1	-0.371	-0.743	0.191
2	1.666	0.542	-0.181
3	0.208	0.617	0.711
4	-0.416	-0.827	0.430
5	-0.003	0.441	0.719



Stochastic Regression Imputation

```
## Generate imputations:  
imps <- imps +  
  rnorm(nrow(yMis), 0, sigma)  
  
## Fill missing cells in Y:  
dat1[mVec, "y"] <- imps  
  
round(head(dat1, n = 5), 3)
```

	y	x	z
1	-0.565	-0.743	0.191
2	1.666	0.542	-0.181
3	0.208	0.617	0.711
4	1.058	-0.827	0.430
5	-0.003	0.441	0.719



Flavors of MI

MI simply repeats a single regression imputation M times.

- The specifics of this regression imputation are important.



Flavors of MI

MI simply repeats a single regression imputation M times.

- The specifics of this regression imputation are important.

Simply repeating the stochastic regression imputation procedure described above won't suffice.

- Still produces too many Type I errors

	Complete Data	PN-Type	PNE-Type
$\text{cor}(X, Y)$	0.499	0.499	0.498
Type I Error	0.040	0.066	0.046

- Type I error rates for PN-Type MI improve on single stochastic regression imputation, but they're still too high.

Proper MI

The problems on the previous slide arise from using the same regression coefficients to create each of the M imputations.

- Implies that you're using the "correct" coefficients.
 - This assumption is plainly ridiculous.
- Proper MI also models uncertainty in the regression coefficients used to create the imputations.
 - A different set of coefficients is randomly sampled (using Bayesian simulation) to create each of the M imputations.
 - The tricky part about implemented MI is deriving the distributions from which to sample these coefficients.

Setting Up Proper MI

Our imputation model is simply a linear regression model:

$$Y = \mathbf{X}\beta + \varepsilon$$

To fully account for model uncertainty, we need to randomly sample both β and $\text{var}(\varepsilon) = \sigma^2$.

- **QUESTION:** Why do we only sample σ^2 and not ε ?



Setting Up Proper MI

Our imputation model is simply a linear regression model:

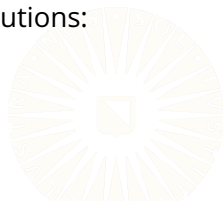
$$Y = \mathbf{X}\beta + \varepsilon$$

To fully account for model uncertainty, we need to randomly sample both β and $\text{var}(\varepsilon) = \sigma^2$.

- **QUESTION:** Why do we only sample σ^2 and not ε ?

For an imputation model with a normally distributed outcome and uninformative priors, we need to specify two distributions:

1. The marginal posterior distribution of σ^2
2. The conditional posterior distribution of β



Marginal Distribution of σ^2

We first specify the marginal posterior distribution for the noise variance, σ^2 .

- This distribution does not depend on any other parameters.

$$\sigma^2 \sim \text{Inv-}\chi^2 (N - P, \text{MSE}) \quad (1)$$

$$\text{with } \text{MSE} = \frac{1}{N - P} \left(Y - \mathbf{X}\hat{\beta}_{ls} \right)^T \left(Y - \mathbf{X}\hat{\beta}_{ls} \right)$$

- σ^2 follows a scaled inverse χ^2 distribution.



Conditional Distribution of β

We then specify the conditional posterior distribution for β .

- This distribution is conditioned on a specific value of σ^2 .

$$\beta \sim \text{MVN} \left(\hat{\beta}_{ls}, \sigma^2 (\mathbf{X}^T \mathbf{X})^{-1} \right) \quad (2)$$

- β (conditionally) follows a multivariate normal distribution.

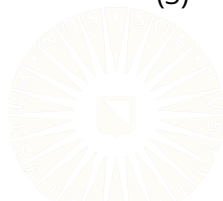


PPD of the Missing Data

Once we've sampled our imputation model parameters, we can construct the posterior predictive distribution of the missing data.

- This is the distribution from which we sample replacements for the missing values (i.e., imputations).
- In practice, we directly compute the imputations based on the simulated imputation model parameters.

$$\begin{aligned} Y_{imp} &= \mathbf{X}_{mis} \tilde{\beta} + \tilde{\varepsilon} \\ \varepsilon &\sim N(\mathbf{0}, \widetilde{\sigma^2}) \end{aligned} \tag{3}$$



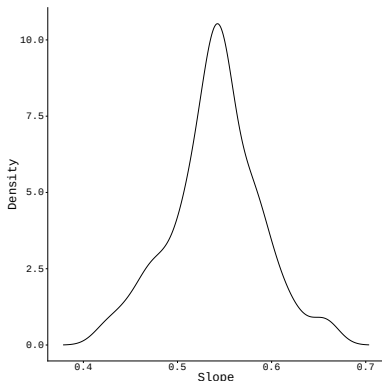
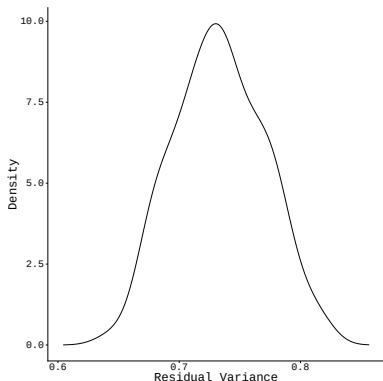
General Steps for Basic MI

With all of the elements in place, we can execute a basic MI by following these steps:

1. Find the least squares estimates of β , $\hat{\beta}_{ls}$, by regressing the observed portion of Y onto the analogous rows of $\mathbf{X}$.
2. Use $\hat{\beta}_{ls}$ to parameterize the posterior distribution of σ^2 , given by Equation 1, and draw M samples of σ^2 from this distribution.
3. For each of the σ_m^2 , sample a corresponding value of β from Equation 2.
4. Plug the M samples of β and σ^2 into Equation 3 to create the M imputations.

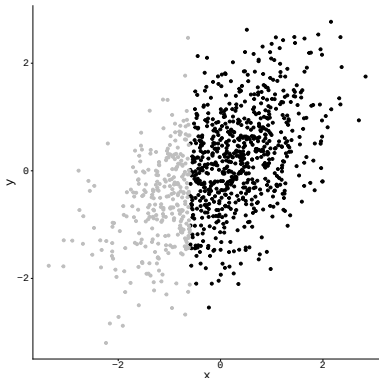
Visualizing MI

Use Bayesian simulation to estimate posterior distributions for the imputation model parameters:



Visualizing MI

Recall the incomplete data from the single imputation examples.



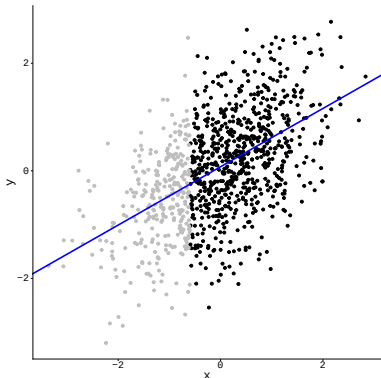
Visualizing MI

Sample values of β_0 and β_1 :

- $\beta_0 = 0.075$
- $\beta_1 = 0.542$

Define the predicted best-fit line:

$$\hat{Y}_{mis} = 0.075 + 0.542X_{mis}$$



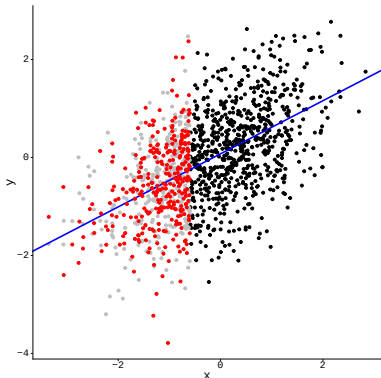
Visualizing MI

Sample a value of σ^2 :

- $\sigma^2 = 0.709$

Generate imputations as in
Stochastic Regression
Imputation:

$$Y_{imp} = \hat{Y}_{mis} + \varepsilon$$
$$\varepsilon \sim N(0, 0.709)$$



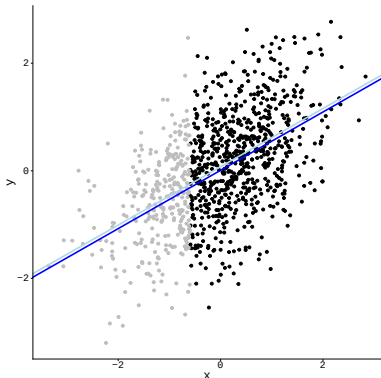
Visualizing MI

Sample values of β_0 and β_1 :

- $\beta_0 = 0.009$
- $\beta_1 = 0.542$

Define the predicted best-fit line:

$$\hat{Y}_{mis} = 0.009 + 0.542X_{mis}$$



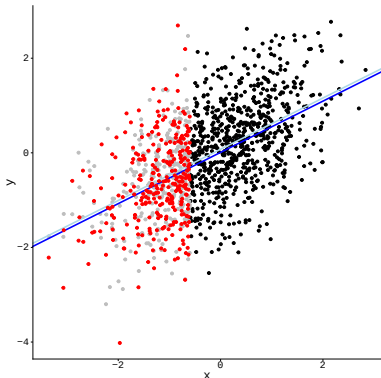
Visualizing MI

Sample a value of σ^2 :

- $\sigma^2 = 0.713$

Generate imputations as in
Stochastic Regression
Imputation:

$$Y_{imp} = \hat{Y}_{mis} + \varepsilon$$
$$\varepsilon \sim N(0, 0.713)$$



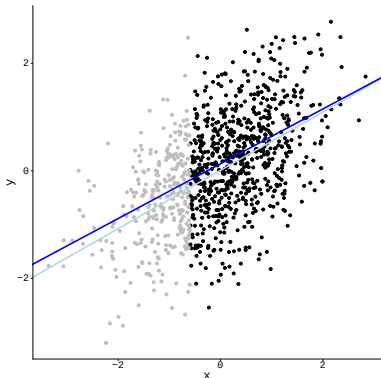
Visualizing MI

Sample values of β_0 and β_1 :

- $\beta_0 = 0.132$
- $\beta_1 = 0.509$

Define the predicted best-fit line:

$$\hat{Y}_{mis} = 0.132 + 0.509X_{mis}$$



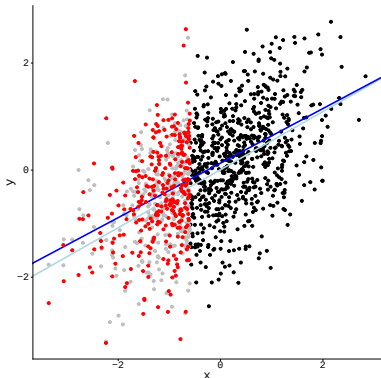
Visualizing MI

Sample a value of σ^2 :

- $\sigma^2 = 0.689$

Generate imputations as in
Stochastic Regression
Imputation:

$$Y_{imp} = \hat{Y}_{mis} + \varepsilon$$
$$\varepsilon \sim N(0, 0.689)$$



Extending the Basic MI Algorithm

The basic MI algorithm described above can be extended in two important ways.

1. Relaxing the normality assumption

- Model the missing data with a non-normal distribution
- Generate imputations from matched, observed cases
- Generate imputations from non-parametric algorithms

2. Imputing multivariate data

- Joint modeling
 - Honaker and King (2010); Rubin (1978, 1987); Schafer (1997)
- Fully conditional specification
 - Raghunathan, Lepkowski, Van Hoewyk, and Solenberger (2001); van Buuren, Brand, Groothuis-Oudshoorn, and Rubin (2006)

Example

```
## Impute the missing data 10 times:
```

```
miceOut <- mice(data    = bfi,  
                m       = 10,  
                maxit    = 10,  
                method   = "pmm",  
                seed     = 235711,  
                print    = FALSE)
```

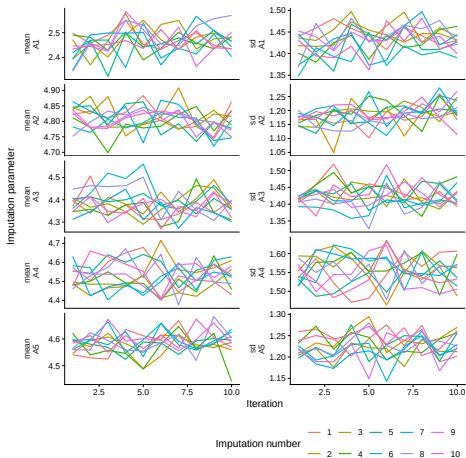
```
## Check convergence via PSR factor:
```

```
rhat(miceOut)
```

	Variable	MissProp	RHat_Mean	RHat_Variance
1	A1	30.21429	1.0097875	1.118038
2	A2	31.46429	0.9937958	1.016108
3	A3	30.17857	1.1010929	1.063382
4	A4	30.03571	1.0307582	1.039938
5	A5	30.75000	1.0201913	1.084875

Example

```
ggmice::plot_trace(miceOut)
```



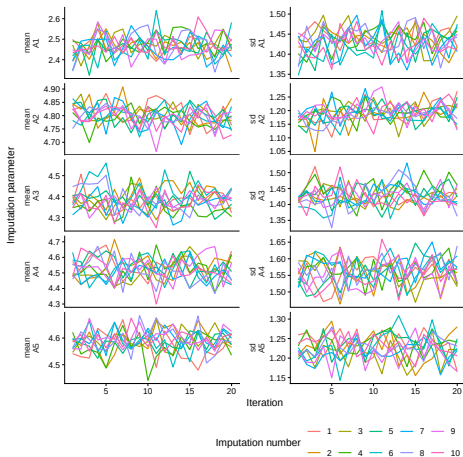
Example

```
## Extend the Markov chains by 10 iterations:  
miceOut <- mice.mids(miceOut, maxit = 10, printFlag = FALSE)  
  
## Check convergence via PSR factor:  
rhat(miceOut)
```

	Variable	MissProp	RHat_Mean	RHat_Variance
1	A1	30.21429	0.9991332	1.0269467
2	A2	31.46429	0.9974385	1.0004546
3	A3	30.17857	1.0990821	1.0404245
4	A4	30.03571	1.0017656	0.9991882
5	A5	30.75000	1.0302434	1.0520755

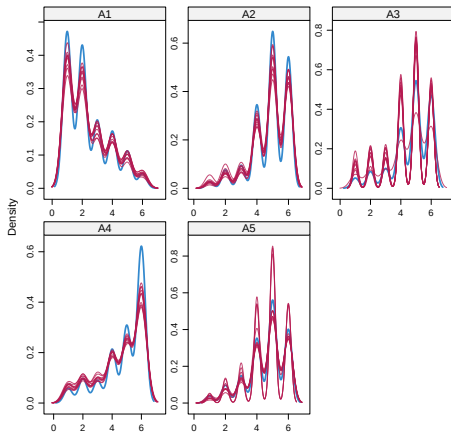
Example

```
ggmice::plot_trace(miceOut)
```



Example

```
mice::densityplot(miceOut)
```



Doing MI-Based Analysis

An MI-based data analysis consists of three phases:

1. The imputation phase

- Replace missing values with M plausible estimates.
- Produce M completed datasets.

2. The analysis phase

- Estimate M replicates of your analysis model.
- Fit the same model to each of the M datasets from Step 1.

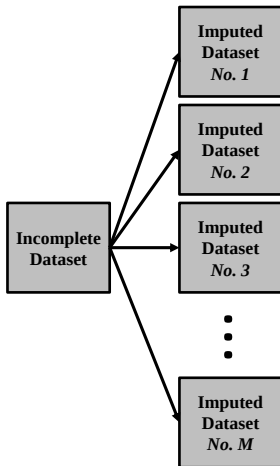
3. The pooling phase

- Combine the M sets of parameter estimates and standard errors from Step 2 into a single set of MI estimates.
- Use these pooled parameter estimates and standard errors for inference.

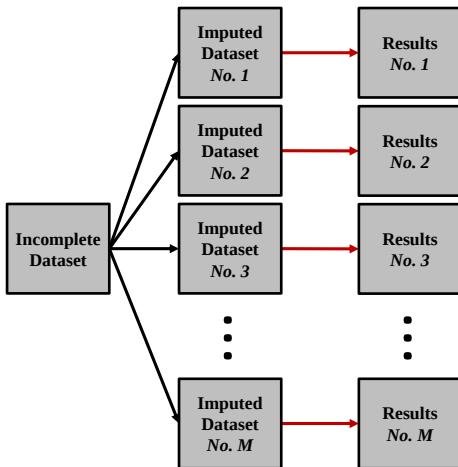
MI-Based Analysis

**Incomplete
Dataset**

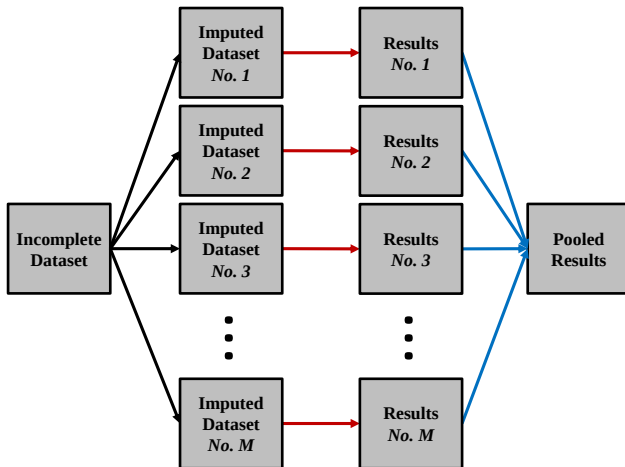
MI-Based Analysis



MI-Based Analysis



MI-Based Analysis



Pooling MI Estimates

Rubin (1987) formulated a simple set of pooling rules for MI estimates.

- The MI point estimate of some interesting quantity, Q^* , is simply the mean of the M estimates, $\{\hat{Q}_m\}$:

$$Q^* = \frac{1}{M} \sum_{m=1}^M \hat{Q}_m$$



Pooling MI Estimates

The MI variability estimate, T , is a slightly more complex entity.

- A weighted sum of the *within-imputation* variance, W , and the *between-imputation* variance, B .

$$W = \frac{1}{M} \sum_{m=1}^M \widehat{SE}_{Q,m}^2$$

$$B = \frac{1}{M-1} \sum_{m=1}^M (\hat{Q}_m - Q^*)^2$$

$$\begin{aligned} T &= W + (1 + M^{-1}) B \\ &= W + B + \frac{B}{M} \end{aligned}$$



Inference with MI Estimates

After computing Q^* and T , we combine them in the usual way to get test statistics and confidence intervals.

$$t = \frac{Q^* - Q_0}{\sqrt{T}}$$

$$CI = Q^* \pm t_{crit} \sqrt{T}$$

We must take care with our df , though.

$$df = (M - 1) \left[1 + \frac{W}{(1 + M^{-1}) B} \right]^2$$



Fraction of Missing Information

The *fraction of missing information* (FMI) is a crucial effect size measure for missing data problems.

$$FMI = \frac{r + \frac{2}{(df+3)}}{r + 1} \approx \frac{(1 + M^{-1})B}{(1 + M^{-1})B + W} \rightarrow \frac{B}{B + W}$$

where

$$r = \frac{(1 + M^{-1})B}{W}$$

The FMI gives us a sense of how much the missing data (and their treatment) have influence our parameter estimates.

- We should report the FMI for an estimated parameter along with other statistics (e.g., t-tests, p-values, effect sizes, etc.).

Example: Analysis & Pooling

We can use the `lavaan.mi()` function (or one of its wrappers) from the **lavaan.mi** package to analyze the multiply imputed datasets and pool results.

```
library(lavaan.mi)

miOut <-
  cfa.mi(cfaMod, data = complete(miceOut, "all"), std.lv = TRUE)
```

Example: Analysis & Pooling

Model Test User Model:

Test statistic	278.946
Degrees of freedom	34
P-value	0.000
Pooling method	D4

Parameter Estimates:

Standard errors	Standard
Information	Expected
Information saturated (h1) model	Structured
Pooled across imputations	Rubin's (1987) rules
Augment within-imputation variance	Scale by average RIV
Wald test for pooled parameters	t(df) distribution

Example: Analysis & Pooling

Latent Variables:

	Estimate	Std.Err	t-value	P(> t)	FMI
agree =~					
A1	0.560	0.034	16.331	0.000	0.303
A2	-0.770	0.027	-28.983	0.000	0.556
A3	-1.014	0.030	-33.803	0.000	0.556
A4	-0.708	0.035	-19.977	0.000	0.399
A5	-0.817	0.028	-28.815	0.000	0.280
open =~					
O1	0.635	0.030	21.272	0.000	0.004
O2	-0.633	0.041	-15.333	0.000	0.014
O3	0.838	0.033	25.309	0.000	0.026
O4	0.344	0.033	10.566	0.000	0.014
O5	-0.641	0.035	-18.367	0.000	0.010

Example: Analysis & Pooling

Covariances:

	Estimate	Std.Err	t-value	P(> t)	FMI
agree ~~					
open	-0.299	0.029	-10.220	0.000	0.305

Example: Analysis & Pooling

Variances:

	Estimate	Std.Err	t-value	P(> t)	FMI
.A1	1.697	0.056	30.215	0.000	0.389
.A2	0.776	0.032	24.268	0.000	0.325
.A3	0.771	0.041	18.707	0.000	0.423
.A4	1.725	0.059	29.171	0.000	0.373
.A5	0.889	0.036	24.420	0.000	0.469
.01	0.878	0.037	23.974	0.000	0.004
.02	2.048	0.071	28.769	0.000	0.008
.03	0.785	0.046	17.043	0.000	0.037
.04	1.369	0.045	30.594	0.000	0.003
.05	1.357	0.050	26.897	0.000	0.008
agree	1.000				
open	1.000				

Model Fit, Model Comparisons, MV Testing

We cannot simply average the χ^2 statistics as we do the parameter estimates.

- We need some fancy processing to get an appropriate statistic.

Three different formulations have been proposed:

- D_1 : Li, Raghunathan, and Rubin (1991)
 - Multivariate Wald-test
- D_2 : Li, Meng, Raghunathan, and Rubin (1991)
 - Direct aggregate of the M test statistics or p-values
- D_3 : Meng and Rubin (1992)
 - Likelihood ratio statistic

See van Buuren (2018, [Section 5.3](#)) for more details.



Model Fit

When we submit the `lavaan.mi` object created by `cfa.mi()` to the `fitMeasures()` function, we can specify the type of pooling rule.

```
what <- c("chisq", "df", "pvalue", "cfi",  
          "rmsea", "rmsea.ci.lower", "rmsea.ci.upper")  
fitMeasures(miOut, fit.measures = what, pool.method = "D3") |>  
  round(3)
```

Test statistic(s) pooled using the D3 pooling method.

Pooled statistic: "standard"

chisq	df	pvalue	cfi
279.070	34.000	0.000	0.909
rmsea	rmsea.ci.lower	rmsea.ci.upper	
0.051	0.045	0.056	

Model Fit

```
fitMeasures(miOut, fit.measures = what, pool.method = "D2") |>  
  round(3)
```

Test statistic(s) pooled using the D2 pooling method.

Pooled statistic: "standard"

chisq	df	pvalue	cfi
243.995	34.000	0.000	0.926
rmsea	rmsea.ci.lower	rmsea.ci.upper	
0.047	0.042	0.053	

Model Comparison

```
## Define a restricted model:
cfaMod2 <- paste(cfaMod, 'agree ~~ 0 * open', sep = '\n')

## Estimate the restricted model and pool the results:
miOut2 <-
  cfa.mi(cfaMod2, data = complete(miceOut, "all"), std.lv = TRUE)

## Test the constraint via nested model comparison:
anova(miOut2, miOut, pool.method = "D3")
```

Test statistic(s) pooled using the D3 pooling method.

Pooled statistic: "standard"

	Df	Chisq	F num	Df den	Df	Pr(>F)	RIV	FMI
miOut	34	279.07						
miOut2	35	353.81	93.96	1	62.196	4.7457e-14	0.32251	0.24386

Model Comparison

```
anova(miOut2, miOut, pool.method = "D2")
```

Test statistic(s) pooled using the D2 pooling method.

Pooled statistic: "standard"

	Df	Chisq	F num	Df	den	Df	Pr(>F)	RIV	FMI
miOut	34	243.995							
miOut2	35	347.227	89.292	1	114.63	5.1039e-16	0.38929	0.28021	

Multivariate Testing

```
## Define a model with some parameter labels:
cfaMod3 <- '
agree =~ 111 * A1 + 121 * A2 + 131 * A3 + 141 * A4 + 151 * A5
open  =~ 112 * O1 + 122 * O2 + 132 * O3 + 142 * O4 + 152 * O5
'

## Estimate the model and pool the results:
miOut3 <-
  cfa.mi(cfaMod3, data = complete(miceOut, "all"), std.lv = TRUE)

## Define some constraints:
cons <- '
111 == 112
121 == 122
131 == 132
141 == 142
151 == 152
'
```


Multivariate Testing

Now, we test the constraints via multivariate Wald tests.

```
lavTestWald.mi(miOut3, constraints = cons, pool.method = "D1")
```

Wald statistic pooled using the D1 pooling method

F	df1	df2	pvalue	ariv	fmi
413.268	5.000	570.423	0.000	0.352	0.260

```
lavTestWald.mi(miOut3, constraints = cons, pool.method = "D2")
```

Wald statistic pooled using the D2 pooling method

F	df1	df2	pvalue	ariv	fmi
435.383	5.000	105.124	0.000	0.298	0.230

ML-BASED SOLUTIONS



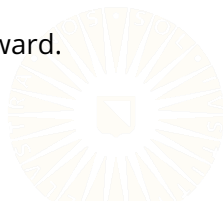
FIML Intuition

FIML is an ML estimation method that is robust to ignorable nonresponse.

- FIML partitions the missing information out of the likelihood function so that the model is only estimated from the observed parts of the data.

After a minor alteration to the likelihood function, FIML reduces to simple ML estimation.

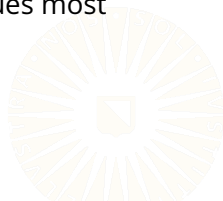
- So, let's review ML estimation before moving forward.



Maximum Likelihood Estimation

ML estimation simply finds the parameter values that are “most likely” to have given rise to the observed data.

- The *likelihood* function is just a probability density (or mass) function with the data treated as fixed and the parameters treated as random variables.
- Having such a framework allows us to ask: “Given that I’ve observed these data values, what parameter values most probably describe these data?”



Maximum Likelihood Estimation

ML estimation is usually employed when there is no closed form solution for the parameters we seek.

- This is why you don't usually see ML used to fit general linear models.

After choosing a likelihood function, we iteratively optimize the function to produce the ML estimated parameters.

- In practice, we nearly always work with the natural logarithm of the likelihood function (i.e., the *loglikelihood*).



ML Intuition

Let's say we have the following $N = 10$ observations.

- We assume these data come from a normal distribution with a known variance of $\sigma^2 = 1$.
- We want to estimate the mean of this distribution, μ .

```
(y <- rnorm(n = 10, mean = 5, sd = 1))
```

```
[1] 4.569057 4.471034 5.158355 4.101269 4.198504  
[6] 4.385373 5.298063 5.717746 5.874183 5.540064
```

ML Intuition

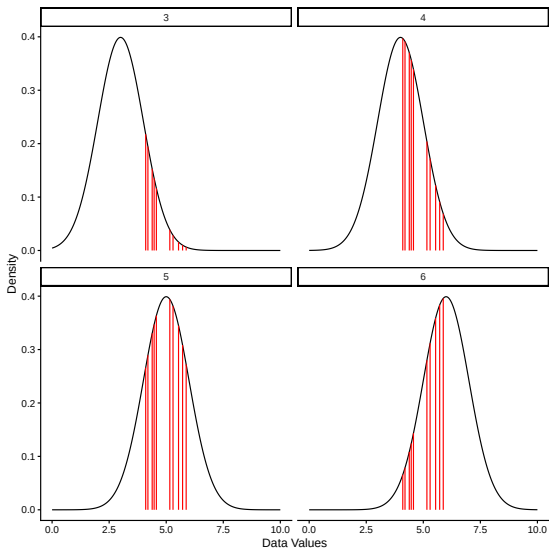
In ML estimation, we would define different normal distributions.

- Every distribution would have $\sigma^2 = 1$.
- Each distribution would have a different value of μ .

We then compare the observed data to those distributions and see which distribution best fits the data.



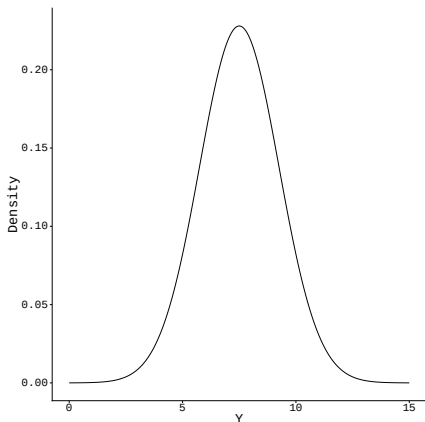
ML Intuition



Likelihoods

Suppose we have the following model:

$$Y \sim N(\mu, \sigma^2).$$



Likelihoods

For a given Y_n , we have:

$$P(Y_n|\mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(Y_n-\mu)^2}{2\sigma^2}}. \quad (4)$$

If we plug estimated parameters into Equation 4, we get the probability of observing Y_n given $\hat{\mu}$ and $\hat{\sigma}^2$:

$$P(Y_n|\hat{\mu}, \hat{\sigma}^2) = \frac{1}{\sqrt{2\pi\hat{\sigma}^2}} e^{-\frac{(Y_n-\hat{\mu})^2}{2\hat{\sigma}^2}}. \quad (5)$$

Applying Equation 5 to all N observations and multiplying the results produces a *likelihood*:

$$L(\hat{\mu}, \hat{\sigma}^2) = \prod_{n=1}^N P(Y_n|\hat{\mu}, \hat{\sigma}^2).$$

Likelihoods

We generally want to work with the natural logarithm of Equation 5. Doing so gives the *loglikelihood*:

$$\begin{aligned}\mathcal{L}(\hat{\mu}, \hat{\sigma}^2) &= \ln \prod_{n=1}^N P(Y_n | \hat{\mu}, \hat{\sigma}^2) \\ &= -\frac{N}{2} \ln 2\pi - N \ln \hat{\sigma} - \frac{1}{2\hat{\sigma}^2} \sum_{n=1}^N (Y_n - \hat{\mu})^2\end{aligned}$$

ML tries to find the values of $\hat{\mu}$ and $\hat{\sigma}^2$ that maximize $\mathcal{L}(\hat{\mu}, \hat{\sigma}^2)$.

- Find the values of $\hat{\mu}$ and $\hat{\sigma}^2$ that are *most likely*, given the observed values of Y .

Likelihoods

Suppose we have a linear regression model:

$$Y = \beta_0 + \beta_1 X + \varepsilon,$$

$$\varepsilon \sim N(0, \sigma^2).$$

This model can be equivalently written as:

$$Y \sim N(\beta_0 + \beta_1 X, \sigma^2)$$

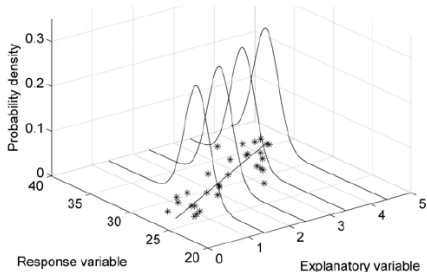


Image retrieved from: <http://www.seaturtle.org/mtn/archives/mtn122/mtn122p1.shtml>

Likelihoods

For a given $\{Y_n, X_n\}$, we have:

$$P(Y_n|X_n, \beta_0, \beta_1, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(Y_n - \beta_0 - \beta_1 X_n)^2}{2\sigma^2}}. \quad (6)$$

If we plug our estimated parameters into Equation 6, we get the probability of observing Y_n given $\hat{Y}_n = \hat{\beta}_0 + \hat{\beta}_1 X_n$ and $\hat{\sigma}^2$.

$$P(Y_n|X_n, \hat{\beta}_0, \hat{\beta}_1, \hat{\sigma}^2) = \frac{1}{\sqrt{2\pi\hat{\sigma}^2}} e^{-\frac{(Y_n - \hat{\beta}_0 - \hat{\beta}_1 X_n)^2}{2\hat{\sigma}^2}} \quad (7)$$

Likelihoods

So, our final loglikelihood function would be the following:

$$\begin{aligned}\mathcal{L}(\hat{\beta}_0, \hat{\beta}_1, \hat{\sigma}^2) &= \ln \prod_{n=1}^N P(Y_n | X_n, \hat{\beta}_0, \hat{\beta}_1, \hat{\sigma}^2) \\ &= -\frac{N}{2} \ln 2\pi - N \ln \hat{\sigma} - \frac{1}{2\hat{\sigma}^2} \sum_{n=1}^N (Y_n - \hat{\beta}_0 - \hat{\beta}_1 X_n)^2.\end{aligned}$$



Example

```
## Fit a model:
out1 <- lm(ldl ~ bp + glu + bmi, data = diabetes)

## Extract the predicted values and the estimated residual
## standard error:
yHat <- predict(out1)
s     <- summary(out1)$sigma

## Compute the row-wise probabilities:
pY <- dnorm(diabetes$ldl, mean = yHat, sd = s)

## Compute the loglikelihood, and compare to R's version:
sum(log(pY)); logLik(out1)[1]

[1] -2109.939
[1] -2109.93
```

Likelihoods

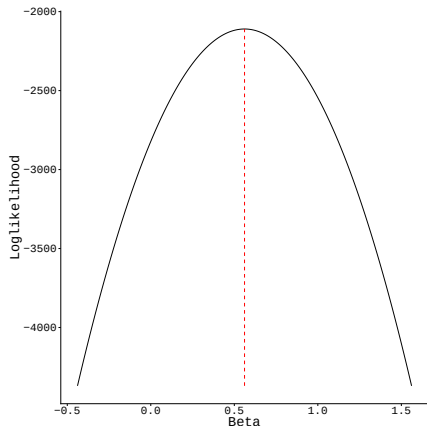
Plugging our estimates into the loglikelihood equation gives:

$$\begin{aligned}\mathcal{L}(\hat{\beta}, \hat{\sigma}^2) &= -221 \times \ln(2\pi) - 442 \times \ln(28.77) \\ &\quad - \frac{\sum_{n=1}^{442} \left(Y_n - 26.59 - 0.08X_n^{bp} - 0.56X_n^{glu} - 1.13X_n^{bmi} \right)^2}{1655} \\ &= -2109.93\end{aligned}$$

Likelihood Function Visualized

Here, we see the conditional loglikelihood function for β_{glu} .

- What loglikelihoods do we get by varying the value of $\hat{\beta}_{glu}$ while keep all other parameters fixed at their ML estimates?



Steps of ML

1. Choose a probability distribution, $f(Y|\theta)$, to describe the distribution of the data, Y , given the parameters, θ .
2. Choose some estimate of θ , $\hat{\theta}^{(i)}$.
3. Compute each row's contribution to the loglikelihood function by evaluating: $\ln \left[f \left(Y_n | \hat{\theta}^{(i)} \right) \right]$.
4. Sum the individual loglikelihood contributions from Step 3 to find the loglikelihood value, $\mathcal{L}^{(i)}$.
5. Choose a “better” estimate of the parameters, $\hat{\theta}^{(i+1)}$, and repeat Steps 3 and 4.
6. Repeat Steps 3 – 5 until the change between $\mathcal{L}^{(i-1)}$ and $\mathcal{L}^{(i)}$ falls below some trivially small threshold.
7. Take $\hat{\theta}^{(i)}$ as your estimate.

Multivariate Normal Distribution

The PDF for the multivariate normal distribution is:

$$P(\mathbf{Y}|\boldsymbol{\mu}, \Sigma) = \frac{1}{\sqrt{(2\pi)^P |\Sigma|}} e^{-\frac{1}{2}(\mathbf{Y}-\boldsymbol{\mu})^T \Sigma^{-1}(\mathbf{Y}-\boldsymbol{\mu})}.$$

So, the multivariate normal loglikelihood is:

$$\mathcal{L}(\boldsymbol{\mu}, \Sigma) = - \left[\frac{P}{2} \ln(2\pi) + \frac{1}{2} \ln |\Sigma| + \frac{1}{2} \right] \sum_{n=1}^N (\mathbf{Y}_n - \boldsymbol{\mu})^T \Sigma^{-1} (\mathbf{Y}_n - \boldsymbol{\mu}).$$

Which can be further simplified if we multiply through by -2:

$$-2\mathcal{L}(\boldsymbol{\mu}, \Sigma) = [P \ln(2\pi) + \ln |\Sigma|] \sum_{n=1}^N (\mathbf{Y}_n - \boldsymbol{\mu})^T \Sigma^{-1} (\mathbf{Y}_n - \boldsymbol{\mu}).$$

ML Example

Recall the n th observation's contribution to the multivariate normal loglikelihood function:

$$\mathcal{L}(\boldsymbol{\mu}, \boldsymbol{\Sigma})_n = -\frac{P}{2} \ln(2\pi) - \frac{1}{2} \ln |\boldsymbol{\Sigma}| - \frac{1}{2} (\mathbf{Y}_n - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1} (\mathbf{Y}_n - \boldsymbol{\mu}).$$

It turns out that this function is readily available in R via the **mvtnorm** package:

```
## Vector of row-wise contributions to the overall LL:  
l10 <- dmvnorm(y, mean = mu, sigma = sigma, log = TRUE)
```

ML Example

We can wrap the preceding code in a nice R function:

```
## Complete data loglikelihood function:
ll <- function(par, data) {
  ## Extract the parameter matrices:
  par <- getParams(par, p = ncol(data))

  ## Compute the row-wise contributions to the LL:
  ll0 <- dmvnorm(data,
                 mean = par$mu,
                 sigma = par$sigma,
                 log = TRUE)

  sum(ll0) # Return the overall LL value
}
```

ML Example

The **optimx** package can numerically optimize arbitrary functions.

- We can use it to (semi)manually implement ML.

```
## Subset the 'diabetes' data:
dat1 <- diabetes |> select(bmi, ldl, glu) |> as.matrix()

## Choose some starting values:
m0    <- rep(0, 3)
s0    <- diag(3) |> vecChol()
par0  <- c(m0, s0)

## Use optimx() to numerically optimize the LL function:
mle <- optimx(par      = par0,
              fn        = ll,
              data      = dat1,
              method     = "BFGS",
              control    = list(maximize = TRUE, maxit = 1000)
              ) |> quiet()
```

ML Example

Check convergence and extract the optimized parameters:

```
## Check convergence:
mle[c("convcode", "kkt1", "kkt2")]

      convcode kkt1 kkt2
BFGS          0 TRUE TRUE

## Get the optimize mean vector and covariance matrix:
muHat    <- mle[1:3]
sigmaHat <- mle[4:9] |>
  as.numeric() |>
  vecChol(p = 3, revert = TRUE)
```

ML Example

Estimate the same parameters using **lavaan::lavCor()**.

```
## Estimate the parameters:
fit <- diabetes |>
  select(bmi, ldl, glu) |>
  lavCor(meanstructure = TRUE, output = "sampstat")

## Extract the estimated parameters:
muHat2    <- fit$mean
sigmaHat2  <- fit$cov
```


ML Example

	bmi	ldl	glu
Manual	26.38	115.44	91.26
Lavaan	26.38	115.44	91.26

Estimated Means

	bmi	ldl	glu		bmi	ldl	glu
bmi	19.48	35.01	19.70	bmi	19.52	35.09	19.74
ldl	35.01	922.82	101.37	ldl	35.09	924.96	101.61
glu	19.70	101.37	131.86	glu	19.74	101.61	132.17

Manual Covariance Matrix

Lavaan Covariance Matrix

From ML to FIML

The n th observation's contribution to the multivariate normal loglikelihood function would be the following:

$$\mathcal{L}(\boldsymbol{\mu}, \boldsymbol{\Sigma})_n = -\frac{P}{2} \ln(2\pi) - \frac{1}{2} \ln |\boldsymbol{\Sigma}| - \frac{1}{2} (\mathbf{Y}_n - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1} (\mathbf{Y}_n - \boldsymbol{\mu}). \quad (8)$$



From ML to FIML

The n th observation's contribution to the multivariate normal loglikelihood function would be the following:

$$\mathcal{L}(\boldsymbol{\mu}, \Sigma)_n = -\frac{P}{2} \ln(2\pi) - \frac{1}{2} \ln |\Sigma| - \frac{1}{2} (\mathbf{Y}_n - \boldsymbol{\mu})^T \Sigma^{-1} (\mathbf{Y}_n - \boldsymbol{\mu}). \quad (8)$$

FIML just tweaks Equation 8 a tiny bit:

$$\mathcal{L}(\boldsymbol{\mu}, \Sigma)_{fiml,n} = -\frac{P}{2} \ln(2\pi) - \frac{1}{2} \ln |\Sigma_q| - \frac{1}{2} (\mathbf{Y}_n - \boldsymbol{\mu}_q)^T \Sigma_q^{-1} (\mathbf{Y}_n - \boldsymbol{\mu}_q).$$

Where $q = 1, 2, \dots, Q$ indexes response patterns.

FIML Example

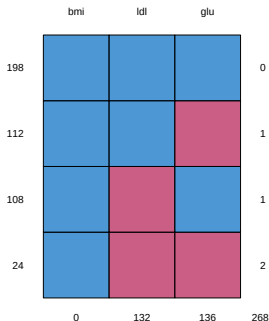
First, punch some holes in our example data.

```
## Impose MAR missing:  
dat2 <- imposeMissData(data    = dat1,  
                        targets = c("ldl", "glu"),  
                        preds   = "bmi",  
                        pm      = 0.3,  
                        types   = c("low", "high"),  
                        stdDat  = TRUE)
```

Visualize the Response Patterns

The data contain 4 unique response patterns.

- We'll define 4 different version of μ and Σ .
- We'll calculate each individual loglikelihood contribution using the appropriate flavor of μ and Σ .



FIML Example

```
## Compute the within-pattern contributions to the LL:
llo <- function(i, mu, sigma, pats, ind, data) {
  ## Define the current response pattern:
  p1 <- pats[i, ]

  if(sum(p1) > 1) # More than one observed variable?
    dmvnorm(x      = data[ind == i, p1],
            mean   = mu[p1],
            sigma  = sigma[p1, p1],
            log    = TRUE)
  else
    dnorm(x      = data[ind == i, p1],
          mean   = mu[p1],
          sd     = sqrt(sigma[p1, p1]),
          log    = TRUE)
}
```

FIML Example

```
## FIML loglikelihood function:
llm <- function(par, data, pats, ind) {
  ## Extract the parameter matrices:
  par <- getParams(par, p = ncol(data))

  ## Compute the pattern-wise contributions to the LL:
  ll1 <- sapply(X      = 1:nrow(pats),
                FUN     = ll0,
                mu      = par$mu,
                sigma   = par$sigma,
                pats    = pats,
                ind     = ind,
                data     = data)

  sum(unlist(ll1))
}
```

FIML Example

```
## Summarize response patterns:
pats <- uniquecombs(!is.na(dat2))
ind  <- attr(pats, "index")

## Choose some starting values:
m0    <- colMeans(dat2, na.rm = TRUE)
s0    <- cov(dat2, use = "pairwise") |> vecChol()
par0  <- c(m0, s0)

## Use optimx() to numerically optimize the LL function:
mle <- optimx(par      = par0,
              fn        = llm,
              data      = dat2,
              pats      = pats,
              ind        = ind,
              method     = "BFGS",
              control    = list(maximize = TRUE, maxit = 1000)
            ) |> quiet()
```


FIML Example

Check convergence and extract the optimized parameters:

```
## Check convergence:
mle[c("convcode", "kkt1", "kkt2")]

      convcode kkt1 kkt2
BFGS          0 TRUE TRUE

## Get the optimize mean vector and covariance matrix:
muHat1    <- mle[1:3]
sigmaHat1 <- mle[4:9] |>
  as.numeric() |>
  vecChol(p = 3, revert = TRUE)
```

FIML Example

To use FIML in **lavaan**, we need only specify `missing = "fiml"`.

- Do the same analysis via **lavaan::cfa()**.

```
## Define the model in lavaan syntax:
mod <- '
bmi ~~ ldl + glu
ldl ~~ glu
'

## Fit the model with lavaan::cfa():
fit <- cfa(mod, data = dat2, missing = "fiml")

## Extract the estimated parameters:
muHat2 <- inspect(fit, "est")$nu
sigmaHat2 <- inspect(fit, "theta")
```

FIML Example

	bmi	ldl	glu
Manual	26.38	115.92	91.22
Lavaan	26.38	115.92	91.22

Estimated Means

	bmi	ldl	glu		bmi	ldl	glu
bmi	19.48	26.05	20.00	bmi	19.48	26.07	20.00
ldl	26.05	968.26	87.52	ldl	26.07	968.33	87.55
glu	20.00	87.52	127.98	glu	20.00	87.55	127.98

Manual Covariance Matrix

Lavaan Covariance Matrix

CFA Example

It is very easy to apply FIML to latent variable models in **lavaan**.

```
## Estimate the CFA from the beginning of the lecture using FIML:  
fimlOut <- cfa(cfaMod, data = bfi, std.lv = TRUE, missing = "fiml")
```

CFA Example

```
partSummary(fimlOut, 1:4)
```

lavaan 0.6-21 ended normally after 41 iterations

Estimator	ML
Optimization method	NLMINB
Number of model parameters	31
Number of observations	2800
Number of missing patterns	32

Model Test User Model:

Test statistic	360.865
Degrees of freedom	34
P-value (Chi-square)	0.000

CFA Example

```
partSummary(fimlOut, 7, fmi = TRUE)
```

Latent Variables:

	Estimate	Std.Err	z-value	P(> z)	FMI
agree =~					
A1	0.538	0.038	14.109	0.000	0.399
A2	-0.764	0.030	-25.530	0.000	0.417
A3	-0.989	0.033	-29.722	0.000	0.434
A4	-0.693	0.039	-17.953	0.000	0.407
A5	-0.838	0.031	-26.622	0.000	0.403
open =~					
O1	0.635	0.025	24.968	0.000	0.003
O2	-0.640	0.036	-17.615	0.000	0.004
O3	0.831	0.029	28.880	0.000	0.008
O4	0.345	0.028	12.333	0.000	0.002
O5	-0.647	0.031	-20.890	0.000	0.005

CFA Example

```
partSummary(fimlOut, 8, fmi = TRUE)
```

Covariances:

	Estimate	Std.Err	z-value	P(> z)	FMI
agree ~~					
open	-0.290	0.028	-10.530	0.000	0.147

CFA Example

```
partSummary(fimlOut, 9, fmi = TRUE)
```

Intercepts:

	Estimate	Std.Err	z-value	P(> z)	FMI
.A1	2.422	0.031	77.128	0.000	0.281
.A2	4.814	0.025	189.726	0.000	0.252
.A3	4.616	0.028	163.443	0.000	0.232
.A4	4.735	0.032	146.063	0.000	0.275
.A5	4.573	0.027	167.561	0.000	0.243
.01	4.816	0.021	225.180	0.000	-0.000
.02	2.713	0.030	91.745	0.000	-0.000
.03	4.441	0.023	192.731	0.000	-0.000
.04	4.894	0.023	212.316	0.000	-0.000
.05	2.490	0.025	99.119	0.000	-0.000

CFA Example

```
partSummary(fimlOut, 10, fmi = TRUE)
```

Variances:

	Estimate	Std.Err	z-value	P(> z)	FMI
.A1	1.693	0.059	28.649	0.000	0.352
.A2	0.765	0.037	20.491	0.000	0.463
.A3	0.736	0.047	15.632	0.000	0.489
.A4	1.654	0.061	27.307	0.000	0.354
.A5	0.876	0.043	20.256	0.000	0.484
.01	0.878	0.031	28.191	0.000	0.003
.02	2.039	0.062	32.972	0.000	0.005
.03	0.797	0.040	19.848	0.000	0.006
.04	1.369	0.038	35.732	0.000	0.002
.05	1.349	0.044	30.369	0.000	0.005
agree	1.000				
open	1.000				

FMI with FIML

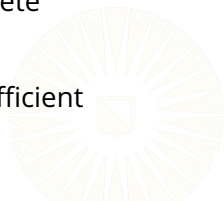
As you saw above, we can also estimate the FMI when using FIML.

- The FMI is calculated using the method described by Savalei and Rhemtulla (2012).

Savalei and Rhemtulla (2012) take an information-theoretic approach to defining the FMI.

- Based on the *Missing Information Principle* of Orchard and Woodbury (1972)
- Their FMI estimates the ratio of missing to complete information for each parameter.

You can use this method to compute the FMI for sufficient statistics via the **semTools::fmi()** function.



AUXILIARY VARIABLES



Satisfying the MAR Assumption

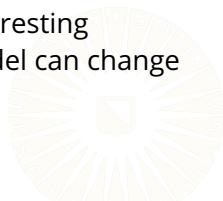
Like MI, FIML also requires MAR data.

- Parameters will be biased when data are MNAR.

Unlike MI, FIML directly treats the missing data while estimating the analysis model.

- The MAR predictors must be included in the analysis model.
- Otherwise, FIML reduces to pairwise deletion.

When the MAR predictors are not substantively interesting variables, naively including them in the analysis model can change the model's meaning.



Saturated Correlates Technique

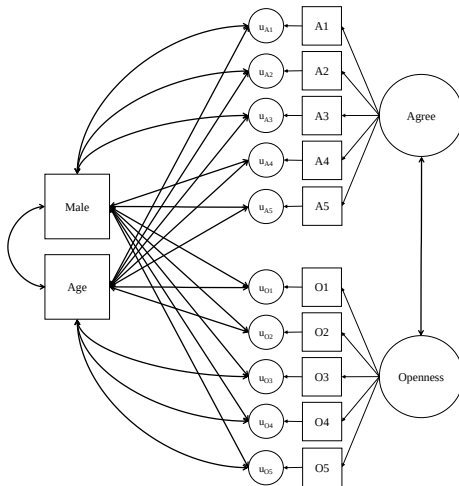
Graham (2003) developed the *saturated correlates* approach to meet two desiderata:

1. Satisfy the MAR assumption by incorporating MAR predictors into the analysis model.
2. Maintain the fit and substantive meaning of the analysis model.

The approach entails incorporating the MAR predictors via a fully-saturated covariance structure:

1. Allow every MAR predictor to covary with all other MAR predictors.
2. Allow every MAR predictor to covary with all observed variables in the analysis model (or their residuals).

Saturated Correlates Diagram



Saturated Correlates Example

We can use the `lavaan.auxiliary()` function from **semTools** (or one of its wrappers) to streamline the analysis.

```
fimlOut2 <- bfi |>
  mutate(male = as.numeric(sex == "male")) |>
  cfa.auxiliary(cfaMod,
    data = _,
    aux = c("age", "male"),
    std.lv = TRUE)
```

Saturated Correlates Example

The `cfa.auxiliary()` function has automatically added the following paths to our model.

```
age ~~ age
```

```
age ~~ male
```

```
male ~~ male
```

```
age ~~ A1
```

```
age ~~ A2
```

```
age ~~ A3
```

```
age ~~ A4
```

```
age ~~ A5
```

```
age ~~ 01
```

```
age ~~ 02
```

```
age ~~ 03
```

```
age ~~ 04
```

```
age ~~ 05
```

```
male ~~ A1
```

```
male ~~ A2
```

```
male ~~ A3
```

```
male ~~ A4
```

```
male ~~ A5
```

```
male ~~ 01
```

```
male ~~ 02
```

```
male ~~ 03
```

```
male ~~ 04
```

```
male ~~ 05
```


Saturated Correlates Example

The auxiliaries have been correlated with all other variables.

```
inspect(fimlOut2, "est")$theta[11:12, 1:6] |> round(3)
```

	A1	A2	A3	A4	A5	O1
age	-2.927	1.330	1.188	2.622	1.863	0.763
male	0.119	-0.094	-0.083	-0.082	-0.073	0.052

```
inspect(fimlOut2, "est")$theta[11:12, 7:12] |> round(3)
```

	O2	O3	O4	O5	age	male
age	-0.773	0.491	0.147	-1.484	123.998	-0.246
male	-0.006	0.020	0.001	-0.015	-0.246	0.220

Saturated Correlates Example

The degrees of freedom have not changed, though.

```
## Naive FIML:
```

```
fitMeasures(fimlOut, "df")
```

```
df
```

```
34
```

```
## FIML w/ saturated correlates:
```

```
fitMeasures(fimlOut2, "df")
```

```
df
```

```
34
```

Saturated Correlates Example

Let's compare the effects of the various missing data treatments on the latent covariance estimates.

	Complete Data	Listwise Deletion	Multiple Imputation	Naive FIML	FIML w/ Auxiliaries
Est	-0.306	-0.254	-0.299	-0.290	-0.295
FMI	—	—	0.305	0.147	0.146

Latent Covariances

References

- Graham, J. W. (2003). Adding missing-data-relevant variables to fimo-based structural equation models. *Structural Equation Modeling: A Multidisciplinary Journal*, 10(1), 80–100. doi: 10.1207/S15328007SEM1001_4
- Honaker, J., & King, G. (2010). What to do about missing values in time-series cross-section data. *American Journal of Political Science*, 54(2), 561–581. doi: 10.1111/j.1540-5907.2010.00447.x
- Li, K.-H., Meng, X.-L., Raghunathan, T. E., & Rubin, D. B. (1991). Significance levels from repeated p-values with multiply-imputed data. *Statistica Sinica*, 1(1), 65–92.
Retrieved from <http://www.jstor.org/stable/24303994>

References

- Li, K. H., Raghunathan, T. E., & Rubin, D. B. (1991). Large-sample significance levels from multiply imputed data using moment-based statistics and an F reference distribution. *Journal of the American Statistical Association*, 86(416), 1065–1073. Retrieved from <http://www.jstor.org/stable/2290525>
- Meng, X.-L., & Rubin, D. B. (1992). Performing likelihood ratio tests with multiply-imputed data sets. *Biometrika*, 79(1), 103–111. Retrieved from <http://www.jstor.org/stable/2337151>
- Orchard, T., & Woodbury, M. A. (1972). A missing information principle: Theory and applications. In *Proceedings of the sixth berkeley symposium on mathematical statistics and probability, volume 1: Theory of statistics*.

References

- Raghunathan, T. E., Lepkowski, J. M., Van Hoewyk, J., & Solenberger, P. (2001). A multivariate technique for multiply imputing missing values using a sequence of regression models. *Survey Methodology*, 27(1), 85–96.
- Rubin, D. B. (1978). Multiple imputations in sample surveys – A phenomenological Bayesian approach to nonresponse. In *Proceedings of the survey research methods section of the American Statistical Association* (Vol. 1, pp. 20–34).
- Rubin, D. B. (1987). *Multiple imputation for nonresponse in surveys* (Vol. 519). New York, NY: John Wiley & Sons.
- Savalei, V., & Rhemtulla, M. (2012). On obtaining estimates of the fraction of missing information from full information maximum likelihood. *Structural Equation Modeling: A Multidisciplinary Journal*, 19(3), 477–494.

References

- Schafer, J. L. (1997). *Analysis of incomplete multivariate data* (Vol. 72). Boca Raton, FL: Chapman & Hall/CRC.
- van Buuren, S. (2018). *Flexible imputation of missing data* (2nd ed.). Boca Raton, FL: CRC Press.
- van Buuren, S., Brand, J. P. L., Groothuis-Oudshoorn, C. G. M., & Rubin, D. B. (2006). Fully conditional specification in multivariate imputation. *Journal of Statistical Computation and Simulation*, 76(12), 1049–1064.

